

Terraform with Azure

Compact Course — Day 3

Scaling, Storage, Databases & Modules

Day 3 — Agenda

Scaling

- Creating Azure Virtual Machine Scale Sets
- Creating Azure Load Balancers
- Autoscale rules, Availability Zones, and rolling upgrades

Storage Accounts

- Azure Blobs, Files, Disks, and Tables
- Access tiers, lifecycle management, and access control

Databases in Azure

- Pre-planning (requirements gathering, scope of work)
- Creating and managing MySQL/MariaDB instances
- PostgreSQL Flexible Server
- Creating and managing Azure SQL Managed Database for MSSQL
- High availability and Private Endpoints

DNS Management with Azure DNS

- Creating and managing domain records
- Private DNS Zones

Modules

- What are modules and why they are powerful
- Module deep dive: inputs, outputs, child modules, and remote modules
- Using `for_each` with modules

Operations & CI/CD

- Terraform workspaces
- CI/CD pipeline patterns
- Best practices and what's next

Part 1 — Scaling

Why Scaling Matters

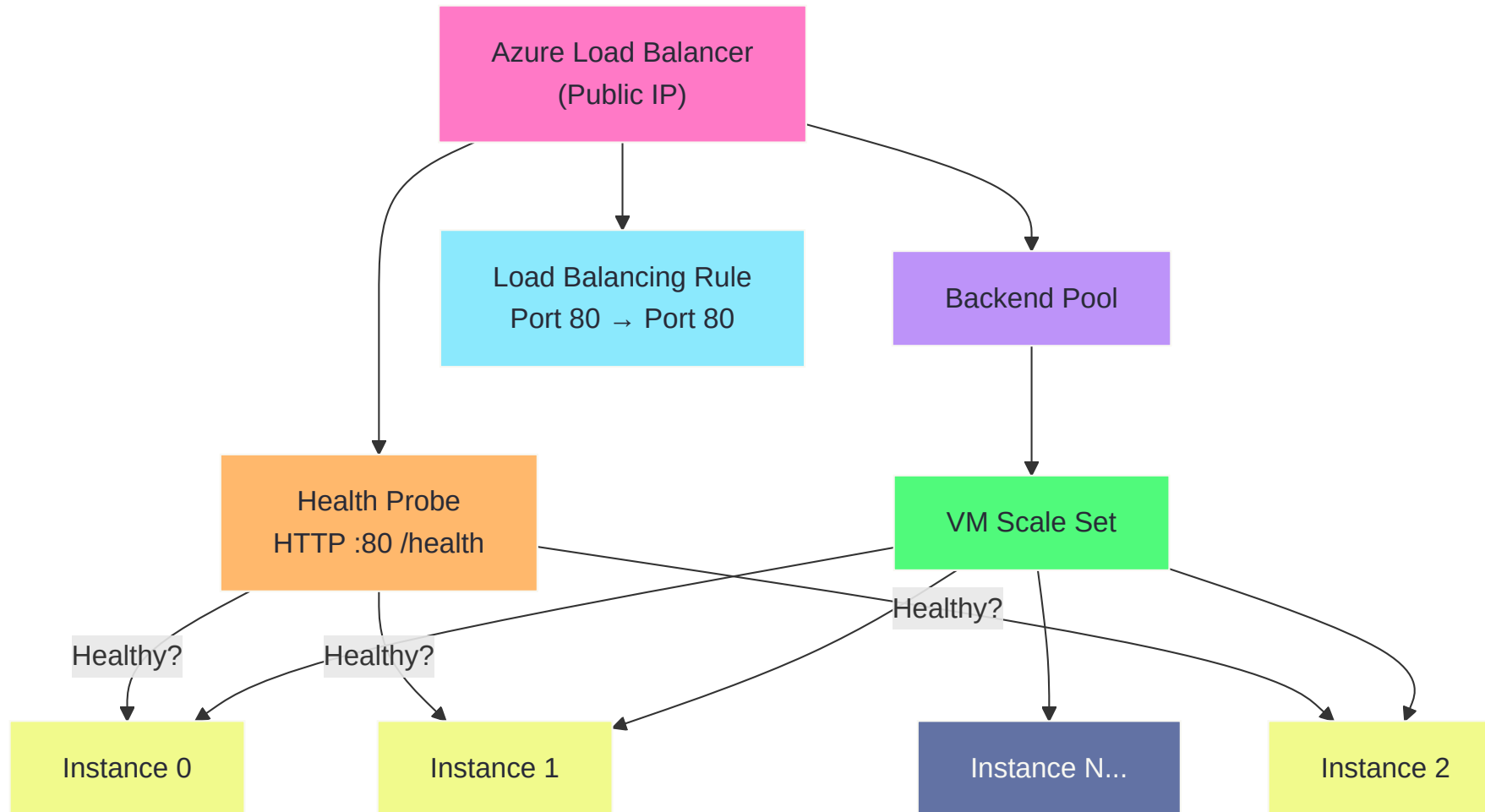
Approach	How	Limitation
Vertical (scale up)	Bigger VM size	Hardware ceiling, downtime
Horizontal (scale out)	More VM instances	Need a load balancer

Azure provides two key resources for horizontal scaling:

- **Virtual Machine Scale Sets (VMSS)** — manage N identical VMs as a group
- **Azure Load Balancer** — distribute traffic across the instances

Terraform makes scaling declarative: change `instances = 2` to `instances = 5`, run `apply`, done.

VMSS + Load Balancer Architecture



Load Balancer — Components

Component	Terraform Resource	Purpose
Public IP	<code>azurerm_public_ip</code>	Entry point for traffic
Load Balancer	<code>azurerm_lb</code>	The LB itself
Frontend IP	<code>azurerm_lb</code> (inline)	Binds the public IP
Backend Pool	<code>azurerm_lb_backend_address_pool</code>	Group of VMs to receive traffic
Health Probe	<code>azurerm_lb_probe</code>	Checks if instances are healthy
LB Rule	<code>azurerm_lb_rule</code>	Maps frontend port → backend port

Creating the Load Balancer

```
resource "azurerm_public_ip" "lb" {
  name                        = "pip-${var.environment}-lb"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  allocation_method          = "Static"
  sku                        = "Standard"
}

resource "azurerm_lb" "main" {
  name                        = "lb-${var.environment}-web"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  sku                        = "Standard"

  frontend_ip_configuration {
    name                        = "frontend"
    public_ip_address_id       = azurerm_public_ip.lb.id
  }
}
```

Backend Pool, Health Probe, and Rule

```
resource "azurerm_lb_backend_address_pool" "main" {
  loadbalancer_id = azurerm_lb.main.id
  name            = "backend-pool"
}

resource "azurerm_lb_probe" "http" {
  loadbalancer_id = azurerm_lb.main.id
  name            = "http-probe"
  protocol        = "Http"
  port            = 80
  request_path     = "/health"
}

resource "azurerm_lb_rule" "http" {
  loadbalancer_id      = azurerm_lb.main.id
  name                 = "http-rule"
  protocol              = "Tcp"
  frontend_port        = 80
  backend_port          = 80
  frontend_ip_configuration_name = "frontend"
  backend_address_pool_ids = [azurerm_lb_backend_address_pool.main.id]
  probe_id              = azurerm_lb_probe.http.id
}
```

Creating a VM Scale Set

```
resource "azurerm_linux_virtual_machine_scale_set" "web" {
  name                = "vmss-${var.environment}-web"
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  sku                 = var.vm_size
  instances           = var.instance_count # e.g., 2
  admin_username      = "adminuser"

  admin_ssh_key {
    username   = "adminuser"
    public_key = file("~/.ssh/id_rsa.pub")
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  network_interface {
    name        = "nic-vmss"
    primary     = true
    ip_configuration {
      name                          = "internal"
      primary                       = true
      subnet_id                    = azurerm_subnet.subnets["web"].id
      load_balancer_backend_address_pool_ids = [azurerm_lb_backend_address_pool.main.id]
    }
  }
}
```

VMSS — Custom Data (Bootstrap Script)

```
resource "azurerm_linux_virtual_machine_scale_set" "web" {  
  # ... (all previous config)  
  
  custom_data = base64encode(<<-EOF  
    #!/bin/bash  
    apt-get update -y  
    apt-get install -y nginx  
    echo "<h1>Hello from $(hostname)</h1>" > /var/www/html/index.html  
    systemctl enable nginx  
    systemctl start nginx  
  EOF  
)  
}
```

`custom_data` runs **once** when the instance is first created. Use it for bootstrapping packages and basic configuration.

Verify

```
$ curl http://<load-balancer-public-ip>  
<h1>Hello from vmss-dev-web_0</h1>
```

Scaling — Just Change a Variable

```
# variables.tf
variable "instance_count" {
  type    = number
  default = 2
}
```

```
# Scale from 2 → 5 instances
$ terraform apply -var="instance_count=5"

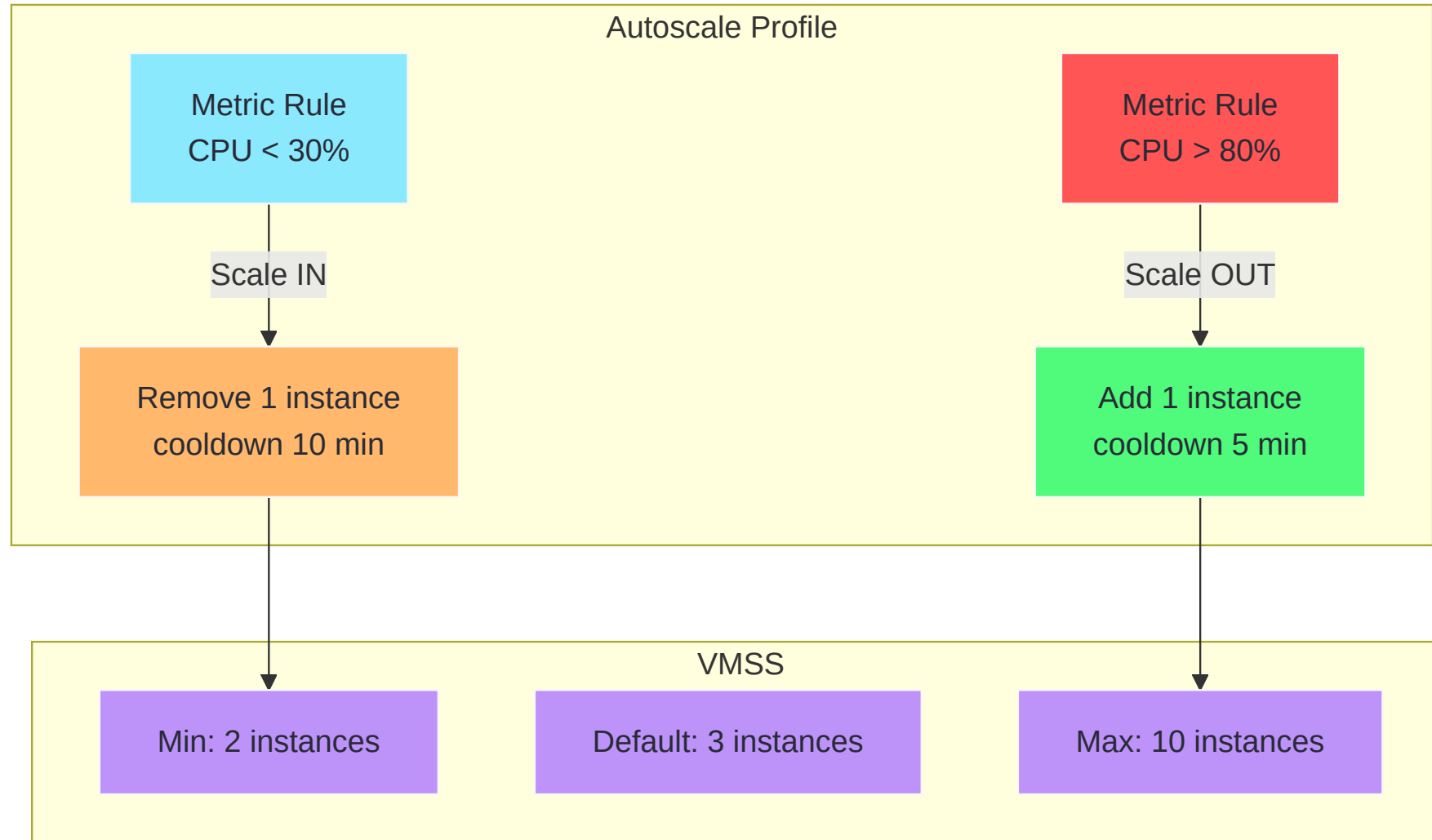
# azurerm_linux_virtual_machine_scale_set.web will be updated in-place
~ resource "azurerm_linux_virtual_machine_scale_set" "web" {
  ~ instances = 2 -> 5
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

The scale set is updated **in-place** — existing instances are preserved, new ones

Autoscale Rules



Autoscale in Terraform

```
resource "azurerm_monitor_autoscale_setting" "vmss" {
  name = "autoscale-${var.environment}-web"
  resource_group_name = azurerm_resource_group.main.name
  location = azurerm_resource_group.main.location
  target_resource_id = azurerm_linux_virtual_machine_scale_set.web.id

  profile {
    name = "default"
    capacity {
      default = 2
      minimum = 2
      maximum = 10
    }

    rule {
      metric_trigger {
        metric_name = "Percentage CPU"
        metric_resource_id = azurerm_linux_virtual_machine_scale_set.web.id
        time_grain = "PT1M"
        statistic = "Average"
        time_window = "PT5M"
        time_aggregation = "Average"
        operator = "GreaterThan"
        threshold = 80
      }
      scale_action {
        direction = "Increase"
        type = "ChangeCount"
        value = "1"
        cooldown = "PT5M"
      }
    }

    rule {
      metric_trigger {
        metric_name = "Percentage CPU"
        metric_resource_id = azurerm_linux_virtual_machine_scale_set.web.id
        time_grain = "PT1M"
        statistic = "Average"
        time_window = "PT10M"
        time_aggregation = "Average"
        operator = "LessThan"
        threshold = 30
      }
      scale_action {
        direction = "Decrease"
        type = "ChangeCount"
        value = "1"
        cooldown = "PT10M"
      }
    }
  }
}
```


Autoscale — Key Parameters

Parameter	Description
<code>capacity.minimum</code>	Never go below this instance count
<code>capacity.maximum</code>	Never exceed this instance count
<code>capacity.default</code>	Starting count when no metrics exist
<code>time_window</code>	How far back to look for the metric
<code>cooldown</code>	Wait time after scaling before evaluating again

Best practices

- **Scale out aggressively**, scale in conservatively (asymmetric cooldowns)
- Use **multiple rules**: CPU + memory + queue depth
- Set **notifications** on scale events for visibility
- Test with load-generation tools (`hey` , `wrk` , `k6`)

VMSS — Availability Zones

```
resource "azurerm_linux_virtual_machine_scale_set" "web" {  
  name                = "vmss-${var.environment}-web"  
  resource_group_name = azurerm_resource_group.main.name  
  location            = azurerm_resource_group.main.location  
  sku                 = var.vm_size  
  instances           = var.instance_count  
  zones               = ["1", "2", "3"] # spread across all zones  
  zone_balance        = true           # balance evenly  
  
  # ... (rest of config)  
}
```

Attribute	Effect
zones	Which availability zones to use
zone_balance	Distribute instances evenly across zones

Combining **autoscale + availability zones** gives you both elasticity and fault tolerance.

VMSS — Rolling Upgrade Policy

```
resource "azurerm_linux_virtual_machine_scale_set" "web" {  
  # ...  
  upgrade_mode = "Rolling"  
  
  rolling_upgrade_policy {  
    max_batch_instance_percent           = 20  
    max_unhealthy_instance_percent      = 20  
    max_unhealthy_upgraded_instance_percent = 5  
    pause_time_between_batches          = "PT2M"  
  }  
  
  health_probe_id = azurerm_lb_probe.http.id  
}
```

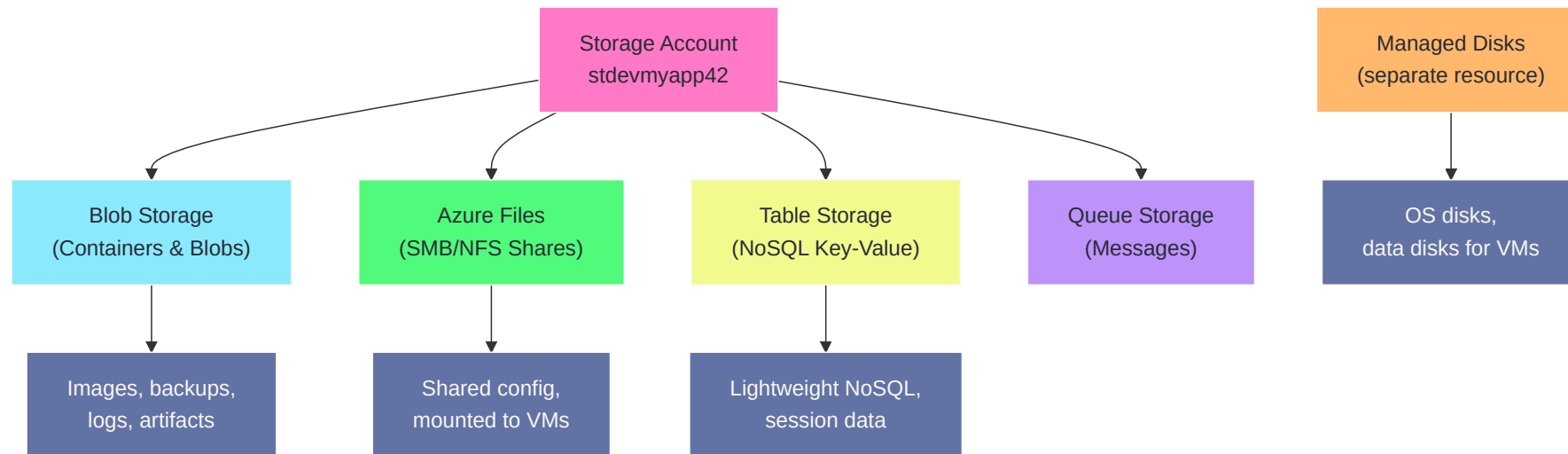
Upgrade modes

Mode	Behavior
Manual	You manually upgrade instances after config change
Automatic	All instances upgraded simultaneously
Rolling	Upgraded in batches with health checks

Use **Rolling** for production to avoid downtime during image or config updates.

Part 2 — Storage Accounts

Azure Storage — Overview



Storage Account — The Parent Resource

```
resource "azurerm_storage_account" "main" {
  name                        = "st${var.environment}myapp42"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
  min_tls_version             = "TLS1_2"

  tags = {
    environment = var.environment
  }
}
```

Key attributes

Attribute	Options	Notes
account_tier	Standard , Premium	Premium for low-latency

Azure Blobs — Object Storage

```
resource "azurerm_storage_container" "data" {
  name                        = "data"
  storage_account_id        = azurerm_storage_account.main.id
  container_access_type     = "private"
}

resource "azurerm_storage_blob" "sample" {
  name                      = "hello.txt"
  storage_account_name     = azurerm_storage_account.main.name
  storage_container_name   = azurerm_storage_container.data.name
  type                     = "Block"
  source_content            = "Hello from Terraform!"
}
```

Blob types

Type	Use case
Block	Text, images, logs, backups (most common)
Append	Log files that grow over time
Page	VHD files for VMs (random read/write)

Azure Files — Shared File System

```
resource "azurerm_storage_share" "config" {  
  name                = "config-share"  
  storage_account_id = azurerm_storage_account.main.id  
  quota               = 5   # GB  
}
```

Use cases

- Mount to **VMs** via SMB (`\\stdevmyapp42.file.core.windows.net/config-share`)
- Mount to **Azure Kubernetes Service** pods
- Shared configuration, log aggregation, home directories

Mount to a Linux VM

```
sudo mount -t cifs \  
//stdevmyapp42.file.core.windows.net/config-share \  
/mnt/config \  
-o vers=3.0,username=stdevmyapp42,password=<key>,dir_mode=0777,file_mode=0777
```

Azure Managed Disks

```
resource "azurerm_managed_disk" "data" {
  name                        = "disk-${var.environment}-data"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  storage_account_type       = "Premium_LRS"
  create_option               = "Empty"
  disk_size_gb               = 64
}

# Attach to a VM
resource "azurerm_virtual_machine_data_disk_attachment" "data" {
  managed_disk_id            = azurerm_managed_disk.data.id
  virtual_machine_id         = azurerm_linux_virtual_machine.web.id
  lun                        = 0
  caching                     = "ReadWrite"
}
```

Disk type	Performance	Use case
Standard_LRS	HDD, up to 500 IOPS	Dev/test, backups
StandardSSD_LRS	SSD, up to 6000 IOPS	Light workloads
Premium_LRS	SSD, up to 20k IOPS	Production databases
UltraSSD_LRS	SSD, up to 160k IOPS	Extreme IO workloads

Azure Table Storage

```
resource "azurerm_storage_table" "sessions" {  
  name                = "sessions"  
  storage_account_id = azurerm_storage_account.main.id  
}
```

Key facts

- Simple **NoSQL key-value** store (PartitionKey + RowKey)
- Good for: session state, user profiles, device metadata
- **Not** a replacement for Cosmos DB (no secondary indexes, limited querying)
- Very cost-effective for simple lookup patterns

When to use Table Storage vs. Cosmos DB vs. SQL

Need	Use
Simple key-value lookups	Table Storage
Global distribution, rich queries	Cosmos DB
Relational data, complex joins	Azure SQL

Storage Account — Security Best Practices

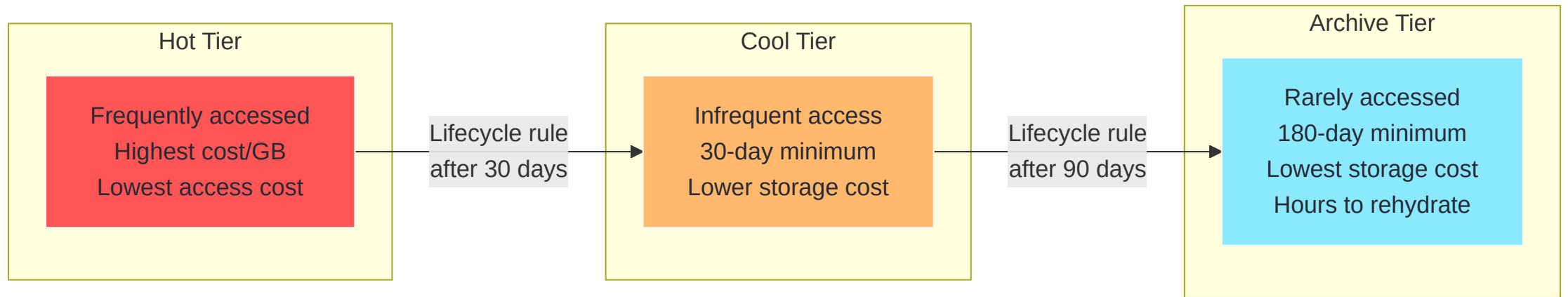
```
resource "azurerm_storage_account" "main" {
  name                        = "st${var.environment}myapp42"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"

  # Security settings
  min_tls_version             = "TLS1_2"
  https_traffic_only_enabled  = true
  public_network_access_enabled = true      # set false + private endpoint in prod
  allow_nested_items_to_be_public = false

  blob_properties {
    delete_retention_policy {
      days = 7
    }
  }
}
```

In production: disable public access and use **Private Endpoints** for secure

Blob Access Tiers



Access Tiers in Terraform

```
resource "azurerm_storage_account" "main" {  
  name                        = "st${var.environment}myapp42"  
  resource_group_name        = azurerm_resource_group.main.name  
  location                   = azurerm_resource_group.main.location  
  account_tier                = "Standard"  
  account_replication_type    = "LRS"  
  access_tier                 = "Hot"      # Default access tier for blobs  
}
```

Tier	Storage cost	Access cost	Min retention	Use case
Hot	Highest	Lowest	None	Frequently accessed data
Cool	Lower	Higher	30 days	Infrequent access, backups
Archive	Lowest	Highest	180 days	Compliance, long-term backup

Set per-account default with `access_tier` , or override per-blob.

Lifecycle Management

```
resource "azurerm_storage_management_policy" "lifecycle" {
  storage_account_id = azurerm_storage_account.main.id

  rule {
    name      = "move-to-cool"
    enabled   = true
    filters {
      blob_types    = ["blockBlob"]
      prefix_match  = ["logs/"]
    }
    actions {
      base_blob {
        tier_to_cool_after_days_since_modification_greater_than    = 30
        tier_to_archive_after_days_since_modification_greater_than = 90
        delete_after_days_since_modification_greater_than          = 365
      }
      snapshot {
        delete_after_days_since_creation_greater_than = 30
      }
    }
  }
}
```

Storage Access Control

Three approaches

Method	Best for	Granularity
Access Keys	Quick/dirty, development	Full account access
SAS Tokens	Time-limited, scoped access	Container/blob level
Azure RBAC	Production, managed identities	Fine-grained roles

RBAC example in Terraform

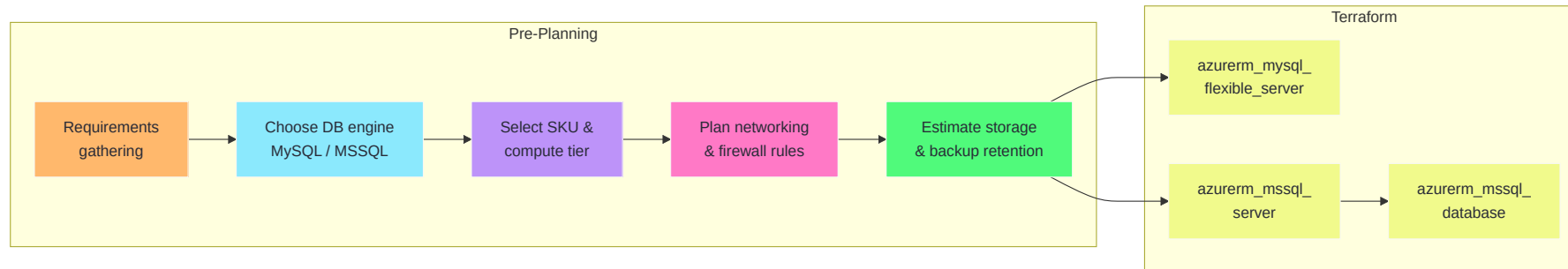
```
resource "azurerm_role_assignment" "blob_reader" {  
  scope          = azurerm_storage_account.main.id  
  role_definition_name = "Storage Blob Data Reader"  
  principal_id    = azurerm_linux_virtual_machine.web.identity[0].principal_id  
}
```


SAS via data source

```
data "azurerm_storage_account_sas" "example" {  
  connection_string = azurerm_storage_account.main.primary_connection_string  
  https_only       = true  
  start            = "2024-01-01"  
  expiry           = "2024-12-31"  
  resource_types { container = true; object = true; service = false }  
  services { blob = true; file = false; queue = false; table = false }  
  permissions { read = true; write = false; delete = false; list = true  
               add = false; create = false; update = false; process = false  
               tag = false; filter = false }  
}
```

Part 3 — Databases in Azure

Database Pre-Planning



Database Pre-Planning — Checklist

Before writing any Terraform:

Question	Why it matters
Which engine? MySQL, MariaDB, MSSQL, PostgreSQL	Determines the Terraform resource
SKU / compute tier? Burstable, General Purpose, Business Critical	Cost and performance
Storage size? 20 GB – 16 TB	Hard to shrink later
High availability? Zone-redundant, same-zone	Affects cost and resilience
Backup retention? 1–35 days	Compliance requirements
Network access? Public + firewall, Private endpoint, VNet integration	Security posture

MySQL Flexible Server

```
resource "azurerm_mysql_flexible_server" "main" {
  name                        = "mysql-${var.environment}-app"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  administrator_login        = var.db_admin_username
  administrator_password     = var.db_admin_password # sensitive!
  sku_name                   = "B_Standard_B1ms"
  version                    = "8.0.21"

  storage {
    size_gb = 20
  }

  backup_retention_days = 7

  tags = {
    environment = var.environment
  }
}
```

MySQL — Database and Firewall Rule

```
resource "azurerm_mysql_flexible_database" "app" {
  name                = "appdb"
  resource_group_name = azurerm_resource_group.main.name
  server_name         = azurerm_mysql_flexible_server.main.name
  charset             = "utf8mb4"
  collation           = "utf8mb4_unicode_ci"
}

# Allow Azure services to access the database
resource "azurerm_mysql_flexible_server_firewall_rule" "azure" {
  name                = "AllowAzureServices"
  server_name         = azurerm_mysql_flexible_server.main.name
  resource_group_name = azurerm_resource_group.main.name
  start_ip_address    = "0.0.0.0"
  end_ip_address      = "0.0.0.0"
}

# Allow a specific IP (e.g., your office)
resource "azurerm_mysql_flexible_server_firewall_rule" "office" {
  name                = "AllowOffice"
  server_name         = azurerm_mysql_flexible_server.main.name
  resource_group_name = azurerm_resource_group.main.name
  start_ip_address    = var.office_ip
  end_ip_address      = var.office_ip
}
```

MySQL SKU Options

Tier	SKU example	Use case
Burstable	B_Standard_B1ms	Dev/test, low-traffic apps
General Purpose	GP_Standard_D2ds_v4	Most production workloads
Business Critical	M0_Standard_E2ds_v4	High performance, local SSD

Connection string output

```
output "mysql_fqdn" {  
  value = azurerm_mysql_flexible_server.main.fqdn  
}  
  
output "mysql_connection_string" {  
  value      = "mysql://${var.db_admin_username}@${azurerm_mysql_flexible_server.main.fqdn}:3306/appdb"  
  sensitive = true  
}
```


Azure SQL — Server and Database

```
resource "azurerm_mssql_server" "main" {
  name                        = "sql-${var.environment}-app"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  version                    = "12.0"
  administrator_login        = var.db_admin_username
  administrator_login_password = var.db_admin_password

  minimum_tls_version = "1.2"
}

resource "azurerm_mssql_database" "app" {
  name      = "appdb"
  server_id = azurerm_mssql_server.main.id
  sku_name  = "S0"

  max_size_gb = 2
  collation   = "SQL_Latin1_General_CP1_CI_AS"

  short_term_retention_policy {
    retention_days = 7
  }
}
```

Azure SQL — Firewall and Outputs

```
resource "azurerm_mssql_firewall_rule" "azure" {  
  name          = "AllowAzureServices"  
  server_id     = azurerm_mssql_server.main.id  
  start_ip_address = "0.0.0.0"  
  end_ip_address  = "0.0.0.0"  
}
```

Azure SQL SKU tiers

Model	SKU example	Billing
DTU (simple)	S0 , S1 , P1	Bundled compute + IO
vCore (flexible)	GP_Gen5_2 , BC_Gen5_4	Independent compute + storage
Serverless	GP_S_Gen5_1	Auto-pause, pay per second

```
output "mssql_fqdn" {  
    value = azurerm_mssql_server.main.fully_qualified_domain_name  
}
```

MySQL vs. Azure SQL — Comparison

	MySQL Flexible Server	Azure SQL
Engine	MySQL / MariaDB	Microsoft SQL Server
Terraform resource	<code>azurerm_mysql_flexible_server</code>	<code>azurerm_mssql_server</code> + <code>azurerm_mssql_database</code>
SKU model	Burstable / GP / Business Critical	DTU / vCore / Serverless
Default port	3306	1433
Best for	Open-source stacks (LAMP, WordPress)	.NET apps, enterprise workloads
HA options	Zone-redundant, same-zone	Auto-failover groups, geo-replication

PostgreSQL Flexible Server

```
resource "azurerm_postgresql_flexible_server" "main" {
  name                        = "psql-${var.environment}-app"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  administrator_login        = var.db_admin_username
  administrator_password     = var.db_admin_password
  sku_name                   = "B_Standard_B1ms"
  version                    = "16"
  storage_mb                 = 32768
  backup_retention_days      = 7
  geo_redundant_backup_enabled = false

  zone = "1"
}

resource "azurerm_postgresql_flexible_server_database" "app" {
  name      = "appdb"
  server_id = azurerm_postgresql_flexible_server.main.id
  charset   = "UTF8"
  collation = "en_US.utf8"
}
```

Database High Availability

```
resource "azurerm_postgresql_flexible_server" "main" {
  name                        = "psql-${var.environment}-app"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  sku_name                   = "GP_Standard_D2ds_v4"
  version                    = "16"
  storage_mb                 = 65536

  high_availability {
    mode                        = "ZoneRedundant"
    standby_availability_zone = "2"
  }

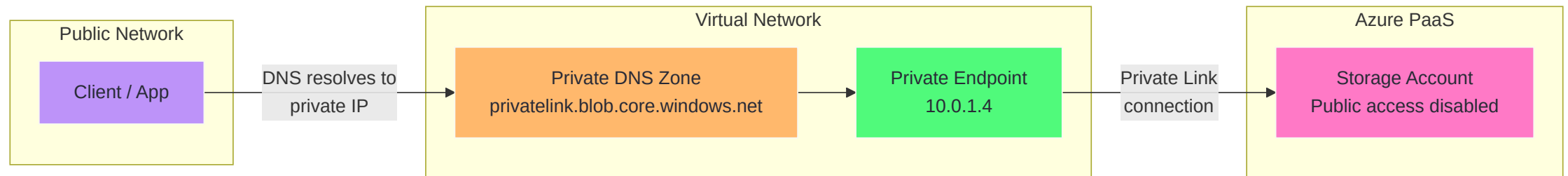
  zone = "1"
  # Primary in Zone 1, standby in Zone 2
  # ...
}
```

HA modes

Mode	Description	Cost
Disabled	Single instance, no failover	1×
SameZone	Standby in same zone (fast failover)	~2×
ZoneRedundant	Standby in different zone (survives zone failure)	~2×

Zone-redundant HA requires **General Purpose** or **Business Critical** SKU.

Private Endpoints for Databases



Private Endpoint in Terraform

```
resource "azurerm_private_endpoint" "db" {
  name                        = "pe-${var.environment}-db"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  subnet_id                  = azurerm_subnet.subnets["db"].id

  private_service_connection {
    name                        = "psc-db"
    private_connection_resource_id = azurerm_postgresql_flexible_server.main.id
    subresource_names          = ["postgresqlServer"]
    is_manual_connection       = false
  }
}

resource "azurerm_private_dns_zone" "db" {
  name                        = "privatelink.postgres.database.azure.com"
  resource_group_name        = azurerm_resource_group.main.name
}

resource "azurerm_private_dns_zone_virtual_network_link" "db" {
  name                        = "link-db"
  resource_group_name        = azurerm_resource_group.main.name
  private_dns_zone_name      = azurerm_private_dns_zone.db.name
  virtual_network_id         = azurerm_virtual_network.main.id
}
```

Part 4 — DNS Management with Azure DNS

Azure DNS — Key Concepts

Concept	Description
DNS Zone	Hosts DNS records for a domain (e.g., <code>example.com</code>)
A Record	Maps a name to an IPv4 address
AAAA Record	Maps a name to an IPv6 address
CNAME Record	Alias for another domain name
MX Record	Mail server routing
TXT Record	Verification, SPF, DKIM
NS Record	Nameserver delegation (auto-created)

Azure DNS is an **authoritative** DNS service — you must point your domain registrar's nameservers to Azure.

Creating a DNS Zone

```
resource "azurerm_dns_zone" "main" {  
  name = "example-training.com"  
  resource_group_name = azurerm_resource_group.main.name  
}  
  
output "dns_nameservers" {  
  value = azurerm_dns_zone.main.name_servers  
}
```

```
$ terraform output dns_nameservers  
[  
  "ns1-01.azure-dns.com.",  
  "ns2-01.azure-dns.net.",  
  "ns3-01.azure-dns.org.",  
  "ns4-01.azure-dns.info.",  
]
```

Update your domain registrar to use these nameservers for the zone to become active.

Creating DNS Records

```
# A record – point "web" to the load balancer IP
resource "azurerm_dns_a_record" "web" {
  name           = "web"
  zone_name      = azurerm_dns_zone.main.name
  resource_group_name = azurerm_resource_group.main.name
  ttl            = 300
  records        = [azurerm_public_ip.lb.ip_address]
}

# CNAME record – alias "www" to "web.example-training.com"
resource "azurerm_dns_cname_record" "www" {
  name           = "www"
  zone_name      = azurerm_dns_zone.main.name
  resource_group_name = azurerm_resource_group.main.name
  ttl            = 300
  record          = "web.example-training.com"
}

# TXT record – domain verification
resource "azurerm_dns_txt_record" "verification" {
  name           = "@"
  zone_name      = azurerm_dns_zone.main.name
  resource_group_name = azurerm_resource_group.main.name
  ttl            = 3600
  record {
    value = "v=spf1 include:spf.protection.outlook.com -all"
  }
}
```

DNS Records — Dynamic with `for_each`

```
variable "dns_a_records" {
  type = map(object({
    ttl      = number
    records = list(string)
  }))
  default = {
    web = { ttl = 300, records = ["20.31.42.53"] }
    api = { ttl = 300, records = ["20.31.42.54"] }
    db  = { ttl = 600, records = ["10.0.3.10"] }
  }
}

resource "azurerm_dns_a_record" "records" {
  for_each = var.dns_a_records

  name           = each.key
  zone_name      = azurerm_dns_zone.main.name
  resource_group_name = azurerm_resource_group.main.name
  ttl            = each.value.ttl
  records        = each.value.records
}
```

Private DNS Zones

For **internal** name resolution within your VNets (no public exposure):

```
resource "azurerm_private_dns_zone" "internal" {
  name                        = "internal.mycompany.com"
  resource_group_name = azurerm_resource_group.main.name
}

resource "azurerm_private_dns_zone_virtual_network_link" "main" {
  name                        = "link-main-vnet"
  resource_group_name       = azurerm_resource_group.main.name
  private_dns_zone_name     = azurerm_private_dns_zone.internal.name
  virtual_network_id        = azurerm_virtual_network.main.id
  registration_enabled      = true # auto-register VM DNS records
}

resource "azurerm_private_dns_a_record" "db" {
  name            = "db"
  zone_name       = azurerm_private_dns_zone.internal.name
  resource_group_name = azurerm_resource_group.main.name
  ttl             = 300
  records         = ["10.0.3.10"]
}
```


Part 5 — Modules

What Are Modules?

A **module** is a directory containing `.tf` files that is used as a **reusable unit**.

```
# You've been using a module all along – the "root" module
project/
├── main.tf           ← root module
├── variables.tf
└── outputs.tf
```

A **child module** is a module called from another module:

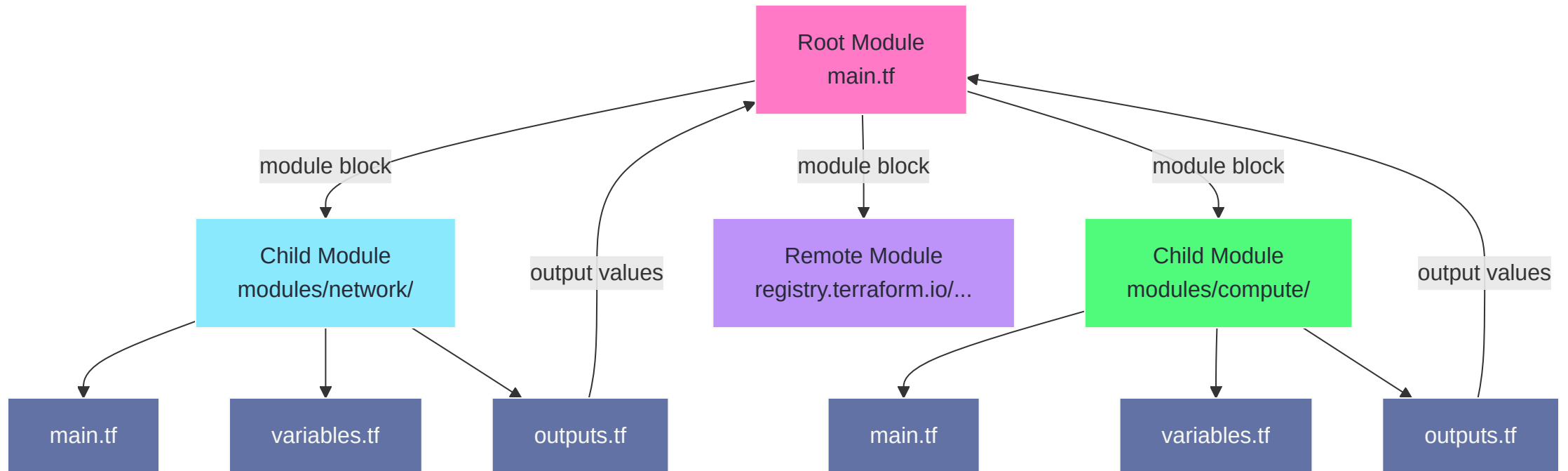
```
project/
├── main.tf                ← root module (calls child modules)
├── modules/
│   ├── network/          ← child module
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── compute/          ← child module
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
```

Why Modules Are So Powerful

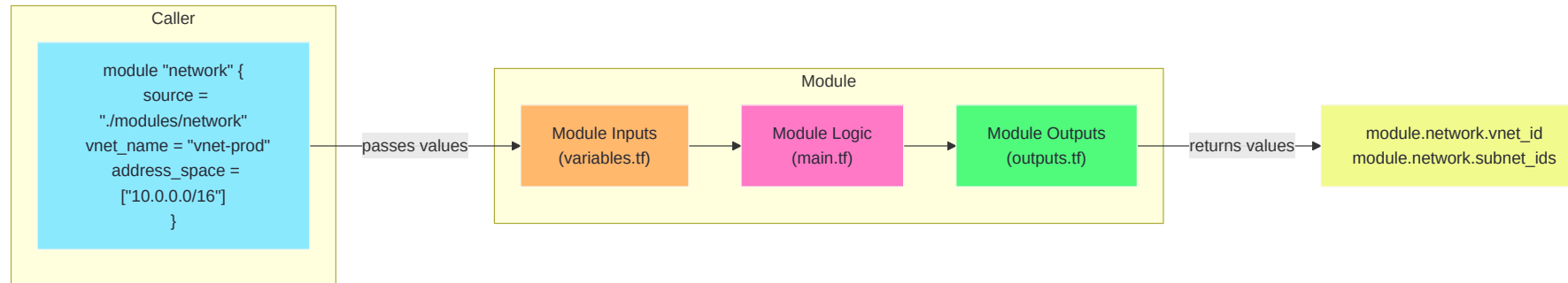
Benefit	Description
Reusability	Write once, use in dev, staging, prod — and across projects
Abstraction	Hide complexity — consumers pass inputs, get outputs
Consistency	Same module = same standards, naming, tagging
Testability	Test a module in isolation, then compose
Composability	Combine small modules into larger architectures
Versioning	Pin module versions, upgrade independently

Without modules, large Terraform projects become copy-paste nightmares.

Module Structure



Module Input / Output Flow



Building a Network Module — `variables.tf`

```
# modules/network/variables.tf

variable "resource_group_name" {
  description = "Name of the resource group"
  type        = string
}

variable "location" {
  description = "Azure region"
  type        = string
}

variable "vnet_name" {
  description = "Name of the virtual network"
  type        = string
}

variable "address_space" {
  description = "VNet address space"
  type        = list(string)
}

variable "subnets" {
  description = "Map of subnet definitions"
  type = map(object({
    address_prefix = string
  }))
}

variable "tags" {
  description = "Tags to apply to all resources"
  type        = map(string)
  default     = {}
}
```

Building a Network Module — `main.tf`

```
# modules/network/main.tf

resource "azurerm_virtual_network" "this" {
  name                = var.vnet_name
  resource_group_name = var.resource_group_name
  location            = var.location
  address_space       = var.address_space
  tags                = var.tags
}

resource "azurerm_subnet" "this" {
  for_each = var.subnets

  name                = each.key
  resource_group_name = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.this.name
  address_prefixes     = [each.value.address_prefix]
}
```


Building a Network Module — `outputs.tf`

```
# modules/network/outputs.tf

output "vnet_id" {
  description = "ID of the virtual network"
  value       = azurerm_virtual_network.this.id
}

output "vnet_name" {
  description = "Name of the virtual network"
  value       = azurerm_virtual_network.this.name
}

output "subnet_ids" {
  description = "Map of subnet name → subnet ID"
  value       = { for k, v in azurerm_subnet.this : k => v.id }
}
```

Return a `map` of subnet IDs so callers can pick by name:

Calling the Module

```
# main.tf (root module)

module "network" {
  source = "../modules/network"

  resource_group_name = azurerm_resource_group.main.name
  location             = azurerm_resource_group.main.location
  vnet_name            = "vnet-${var.environment}-main"
  address_space        = ["10.0.0.0/16"]
  subnets = {
    web = { address_prefix = "10.0.1.0/24" }
    app = { address_prefix = "10.0.2.0/24" }
    db  = { address_prefix = "10.0.3.0/24" }
  }
  tags = var.tags
}

# Use the module's outputs
resource "azurerm_network_interface" "web" {
  name                = "nic-web"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  ip_configuration {
    name          = "primary"
    subnet_id     = module.network.subnet_ids["web"]
    # ...
  }
}
```

Multiple Module Instances

```
# Deploy the same module twice with different parameters

module "network_dev" {
  source = "../modules/network"

  resource_group_name = azurerm_resource_group.dev.name
  location             = "westeurope"
  vnet_name            = "vnet-dev-main"
  address_space        = ["10.0.0.0/16"]
  subnets = {
    web = { address_prefix = "10.0.1.0/24" }
    app = { address_prefix = "10.0.2.0/24" }
  }
  tags = { environment = "dev" }
}

module "network_prod" {
  source = "../modules/network"

  resource_group_name = azurerm_resource_group.prod.name
  location             = "westeurope"
  vnet_name            = "vnet-prod-main"
  address_space        = ["10.1.0.0/16"]
  subnets = {
    web = { address_prefix = "10.1.1.0/24" }
    app = { address_prefix = "10.1.2.0/24" }
    db  = { address_prefix = "10.1.3.0/24" }
  }
  tags = { environment = "prod" }
}
```

Module Inputs — Best Practices

```
variable "vnet_name" {  
  description = "Name of the virtual network" # always document  
  type        = string                       # always type-constrain  
  
  validation {  
    condition      = can(regex("^vnet-", var.vnet_name))  
    error_message = "VNet name must start with 'vnet-'."  
  }  
}
```

Guidelines

- **Every variable** should have `description` and `type`
- **Avoid too many variables** — if your module has 30+ inputs, it's doing too much
- **Provide sensible defaults** where possible
- **Use validation** for naming conventions, allowed values
- **Mark sensitive** variables (passwords, keys) as `sensitive = true`

Handling Module Outputs

Outputs are the module's **public API** — they define what callers can access.

Inside the module

```
output "vnet_id" {  
    value = azurerm_virtual_network.this.id  
}
```

In the caller

```
# Access via module.<NAME>.<OUTPUT>
resource "azurerm_subnet" "extra" {
  virtual_network_name = module.network.vnet_name
  # ...
}

# Pass to another module
module "compute" {
  source      = "../modules/compute"
  subnet_id = module.network.subnet_ids["web"]
}
```

Modules communicate **only** through inputs (variables) and outputs — no global state.

Remote Modules

From the Terraform Registry

```
module "network" {  
  source  = "Azure/network/azurerm"  
  version = "5.3.0"  
  
  resource_group_name = azurerm_resource_group.main.name  
  vnet_name           = "vnet-main"  
  address_space       = "10.0.0.0/16"  
  subnet_prefixes     = ["10.0.1.0/24", "10.0.2.0/24"]  
  subnet_names        = ["web", "app"]  
}
```


From a Git repository

```
module "network" {  
  source = "git::https://github.com/myorg/terraform-modules.git//network?ref=v1.2.0"  
  
  # inputs...  
}
```

Source types

Source	Example
Local path	<code>./modules/network</code>
Terraform Registry	<code>Azure/network/azurerm</code>
GitHub	<code>github.com/org/repo//path</code>
Generic Git	<code>git::https://...?ref=tag</code>
S3 / GCS	<code>s3::https://bucket/path</code>

Module Versioning — Why It Matters

```
module "network" {  
  source = "Azure/network/azurerm"  
  version = "~> 5.0"           # allow 5.x but not 6.0  
}
```

Constraint	Meaning
<code>= 5.3.0</code>	Exact version only
<code>>= 5.0, < 6.0</code>	Any 5.x
<code>~> 5.3</code>	<code>>= 5.3.0</code> and <code>< 5.4.0</code>
<code>~> 5.0</code>	<code>>= 5.0.0</code> and <code>< 6.0.0</code>

Best practices

- **Always pin versions** for remote modules
- Use `~>` (**pessimistic**) to get patch updates but not breaking changes
- **Upgrade deliberately** with a plan + apply cycle
- **Lock file** (`.terraform.lock.hcl`) pins exact versions after `init`

When to Create a Module

Situation	Module?
Same pattern used in 2+ places	✓ Yes
Complex resource with many related resources	✓ Yes
Team-wide standards (naming, tagging, networking)	✓ Yes
One-off resource, used once	✗ No — inline is fine
Wrapping a single resource with no added logic	✗ No — over-abstraction

Good module candidates

- **Network:** VNet + subnets + NSGs
- **Compute:** VM + NIC + disk + public IP
- **Database:** Server + database + firewall rules
- **DNS:** Zone + records

If you're copying and pasting Terraform code, it's time for a module.

for_each with Modules

```
variable "environments" {
  type = map(object({
    location      = string
    address_space = list(string)
    subnets      = map(object({
      address_prefix = string
    }))
  }))
})
default = {
  dev = {
    location      = "westeurope"
    address_space = ["10.0.0.0/16"]
    subnets      = { web = { address_prefix = "10.0.1.0/24" } }
  }
  prod = {
    location      = "westeurope"
    address_space = ["10.1.0.0/16"]
    subnets      = { web = { address_prefix = "10.1.1.0/24" } }
  }
}

module "network" {
  source      = "../modules/network"
  for_each    = var.environments

  resource_group_name = azurerm_resource_group.envs[each.key].name
  location            = each.value.location
  vnet_name            = "vnet-${each.key}-main"
  address_space        = each.value.address_space
  subnets             = each.value.subnets
}
```

Module Composition — Connecting Modules

```
module "network" {
  source = "../modules/network"
  # ... network inputs
}

module "compute" {
  source = "../modules/compute"

  subnet_id          = module.network.subnet_ids["web"]
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  vm_size             = var.vm_size
}

module "monitoring" {
  source = "../modules/monitoring"

  vm_id              = module.compute.vm_id
  resource_group_name = azurerm_resource_group.main.name
  action_group_id    = azurerm_monitor_action_group.ops.id
}
```

Part 6 — Operations & CI/CD

Terraform Workspaces

```
# List workspaces
$ terraform workspace list
* default

# Create and switch to a new workspace
$ terraform workspace new dev
Created and switched to workspace "dev"!

$ terraform workspace new prod
Created and switched to workspace "prod"!

# Switch between workspaces
$ terraform workspace select dev
Switched to workspace "dev".

# Use in code
$ terraform workspace show
dev
```

Workspaces vs. Separate Directories

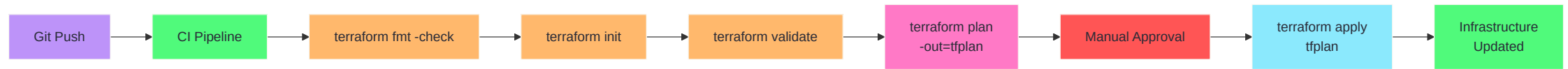
Approach	Workspaces	Separate dirs / repos
State isolation	✓ Per-workspace state	✓ Per-directory state
Code sharing	Same code, different state	Copy or symlink code
Variable differences	Use <code>terraform.workspace</code> + conditionals	Use different <code>.tfvars</code>
Complexity	Simple for few envs	Better for large orgs
Risk	Wrong workspace = wrong env	Harder to mix up

When to use workspaces

-  Small teams, few environments
-  Environments are structurally identical
-  Not for environments that diverge significantly

Many teams prefer **separate directories** (or branches) per environment for safety.

CI/CD Pipeline for Terraform



CI/CD — GitHub Actions Example

```
name: Terraform
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  id-token: write    # OIDC
  contents: read

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.9.0

      - uses: azure/login@v2
        with:
          client-id: ${ secrets.AZURE_CLIENT_ID }
          tenant-id: ${ secrets.AZURE_TENANT_ID }
          subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }

      - run: terraform fmt -check
      - run: terraform init
      - run: terraform validate
      - run: terraform plan -out=tfplan

      - if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: terraform apply tfplan
```

CI/CD — Best Practices

Practice	Why
Always use <code>-out=tfplan</code>	Ensures <code>apply</code> runs the exact reviewed plan
Require PR approval before <code>apply</code>	Human gate for infrastructure changes
Use OIDC, not secrets	No credentials to rotate
Run <code>fmt -check</code> and <code>validate</code>	Catch errors early
Pin Terraform version	Prevent "works on my machine" issues
Comment plan output on PR	Reviewers see what will change
Separate plan and apply jobs	Apply only after approval
Use remote state with locking	Prevent parallel applies

Terraform Best Practices — Summary

Code organization

- One purpose per file: `main.tf`, `variables.tf`, `outputs.tf`, `providers.tf`
- Modules for any pattern used more than once
- Consistent naming: `{type}-{environment}-{purpose}`

Safety

- Always run `plan` before `apply`
- Use `prevent_destroy` on critical resources
- Use remote state with locking
- Mark sensitive variables as `sensitive = true`
- Never commit `.tfstate` or `.tfvars` to Git

Operations

- Pin provider and module versions
- Commit `.terraform.lock.hcl`
- Use `moved` blocks for refactoring
- Import existing resources instead of recreating
- Automate with CI/CD — human approval for `apply`

Day 3 — Recap

What We Covered Today

Scaling

- VMSS for horizontal scaling, Load Balancer with probes and rules
- Autoscale rules with metric triggers and cooldowns
- Availability Zones, zone balancing, and rolling upgrade policies

Storage

- Storage Accounts: Blobs, Files, Managed Disks, Tables
- Access tiers (Hot/Cool/Archive) and lifecycle management policies
- Access control: keys, SAS tokens, and Azure RBAC

Databases

- Pre-planning checklist: engine, SKU, storage, networking, credentials
- MySQL Flexible Server, PostgreSQL Flexible Server, and Azure SQL
- High availability (zone-redundant) and Private Endpoints

DNS

- Public and Private DNS zones, records with `for_each`

Modules

- Module structure, inputs, outputs, composition, and `for_each` with modules
- Remote modules, versioning, and when to create a module

Operations & CI/CD

- Terraform workspaces for state isolation
- CI/CD pipelines (GitHub Actions), best practices

Exercises

See the `exercises/day-3/` folder for four hands-on labs:

1. **Scale Sets and Load Balancer**
2. **Storage Accounts: Blobs, Files, Disks, and Tables**
3. **Databases and DNS Management**
4. **Building a Reusable Terraform Module**



Course Complete!

You are now equipped to independently manage Azure infrastructure with Terraform.

Next steps:

- Practice with real projects
- Explore the Terraform Registry for pre-built modules
- Set up CI/CD pipelines with remote state
- Learn `terraform import` to bring existing resources under management